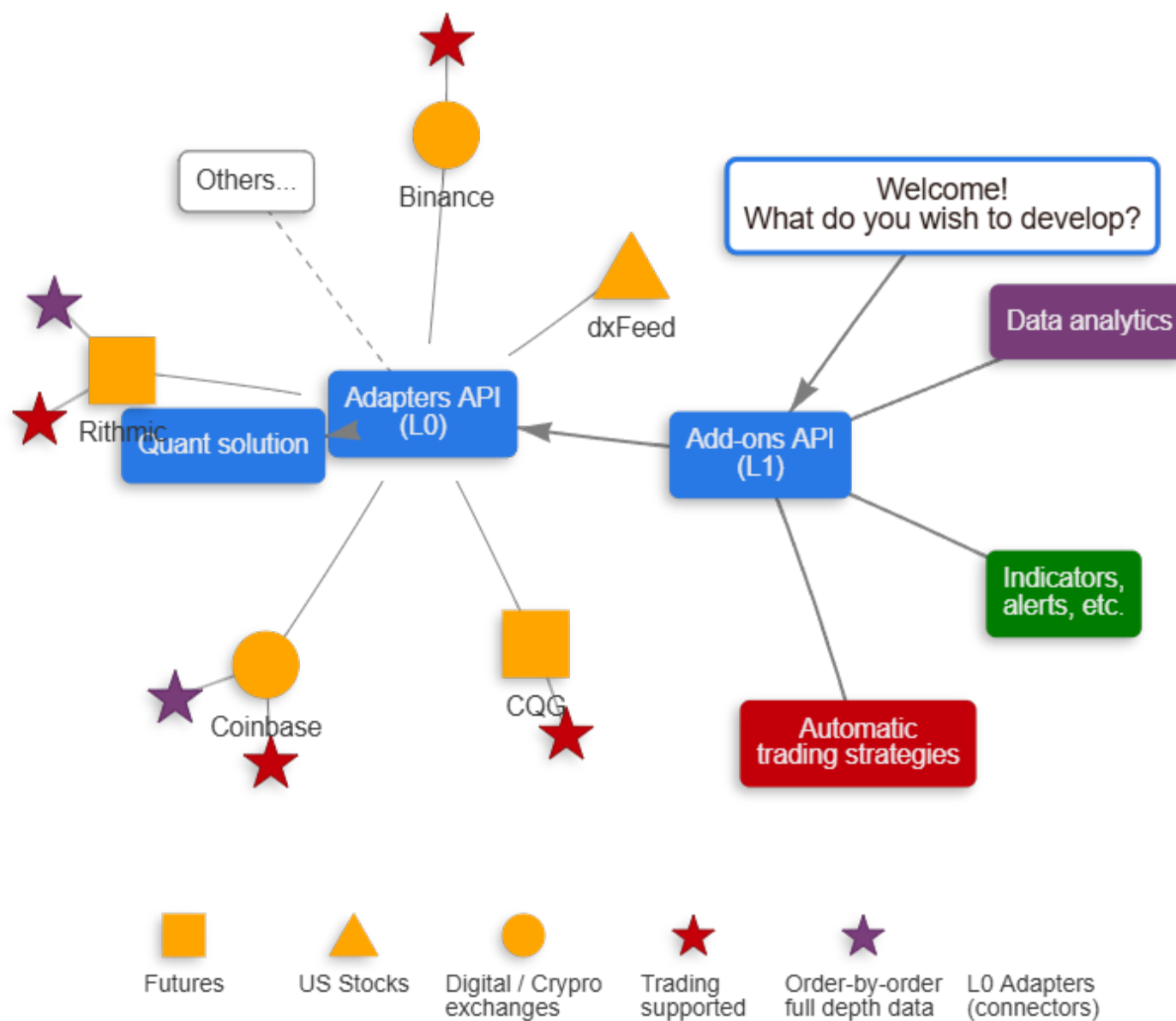


Bookmap API



API modules

Bookmap supports two API types: the Add-ons API (also called the L1 or Layer 1 API) and the Connect API (also called the L0 or the Layer 0 API).

See the below table for differences.

	L1 / Add-ons API	L0 / Connect API
Usage	Create custom add-ons (indicators, trading strategies)	Create custom market data and trading connections
Details	Develop add-ons such as indicators, alerts, trading strategies, etc. Such add-ons can be used in real time and in replay, and in general don't depend on data source type (unless it uses a know-how specific for some data source)	Develop adapters that connect Bookmap to additional market data and/or trading services such as digital/crypto exchanges, market data vendors, brokers, order management systems (OMS), etc.
Extensions	"Intercept / Inject" mode - override upstream and downstream market data and trading messages. See below some of the use cases. Another extension of this API is a "Simplified" development framework which makes add-on development easier. In alpha stage is also our new Python API .	Quant solution - Allows to connect Bookmap to proprietary market data and trading sessions for debriefing, development, real-time monitoring of trading systems. It also includes a collection of useful tools for the quantitative finance industry, especially HFT.

Connect API (L0)

To develop adapters using the L0 API, you must qualify under the Quant solution category. After approval, you can implement your custom L0 API adapters in Bookmap.

Getting Started with L0 API:

- Consult the [readme](#) in the Layer0ApiDemo project.
- Browse examples in the [Layer0ApiDemo](#) project.
- Consider reviewing the [Bitmex adapter](#) implementation.

Add-ons API (L1)

Depending on code complexity and features, there are two types of L1 API:

Simplified API

- Ideal for beginners with many examples.
- Lacks "inject" and "data editor" mode support.
- Available in both Java and Python.

Getting Started:

- Check the [readme](#) in the DemoStrategies project.
- Explore examples in DemoStrategies under [simplified.demo](#) and in the [simple-demo](#) project.
- For Python users, see the [Python API page](#).

Core API:

- More intricate with fewer examples.
- Supports "inject" and "data editor" mode to modify data sent to Bookmap.

- Prior experience with the Simplified API is advised.

Getting Started:

- Read the DemoStrategies project [readme](#).
- View examples in the DemoStrategies [simple-demo](#) sub-package.

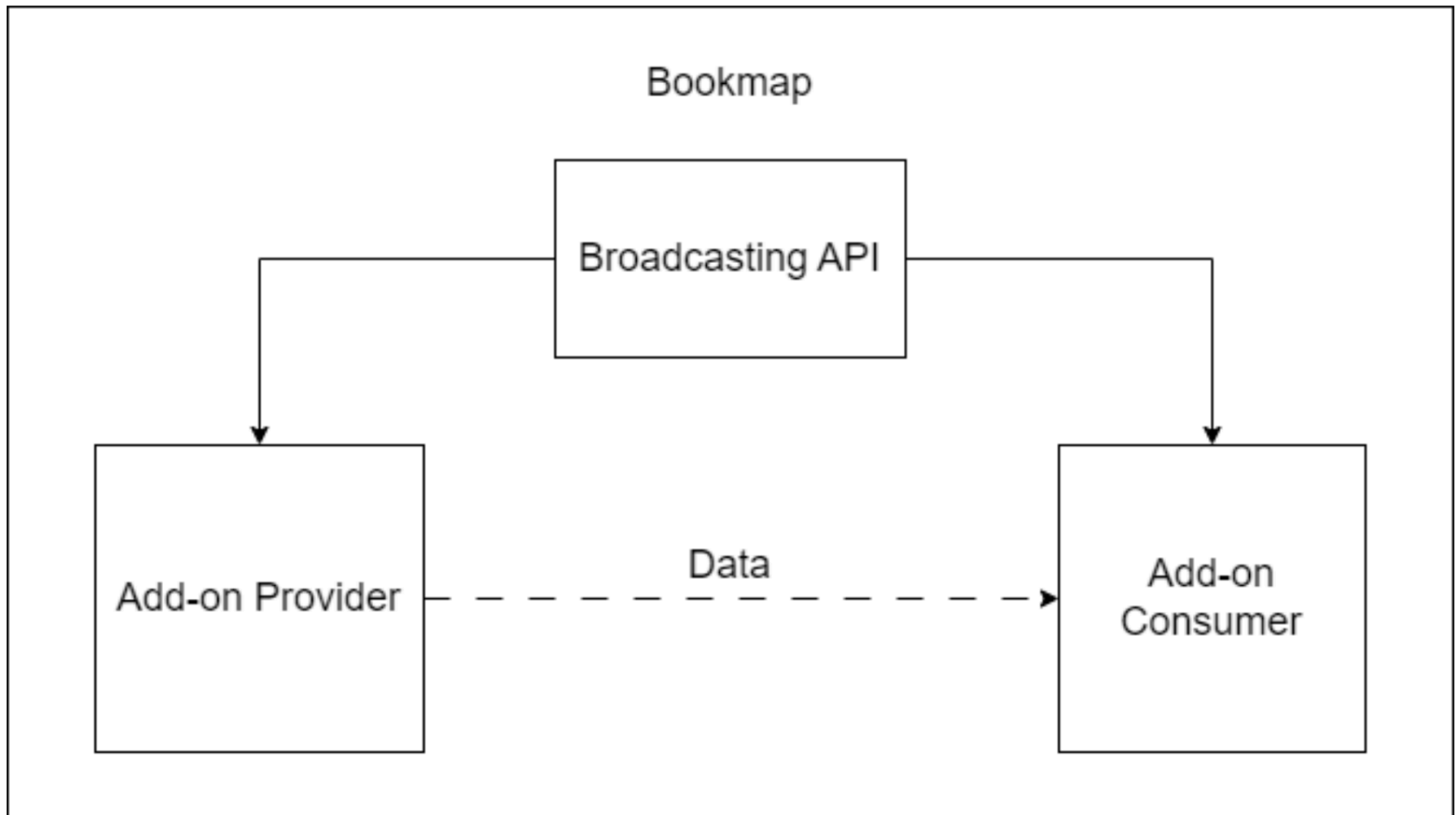
L0 API and L1 API Resources:

- [Repository](#) with documentation and examples.
- [Forum](#) with API sub-forum.
- [API](#) and [Python API](#) channels on Discord.
- [Simplified API videos](#).

Broadcasting API (BrAPI)

The **Broadcasting API** is a protocol that allows data to be **broadcasted** from one add-on to another. It also serves as a library for add-ons using the Java API and is integrated into the Python API, but only for consuming live events. There are two types of add-ons in Broadcasting:

- Providers are add-ons that **broadcast** data.
- Consumers are add-ons that receive data from providers.

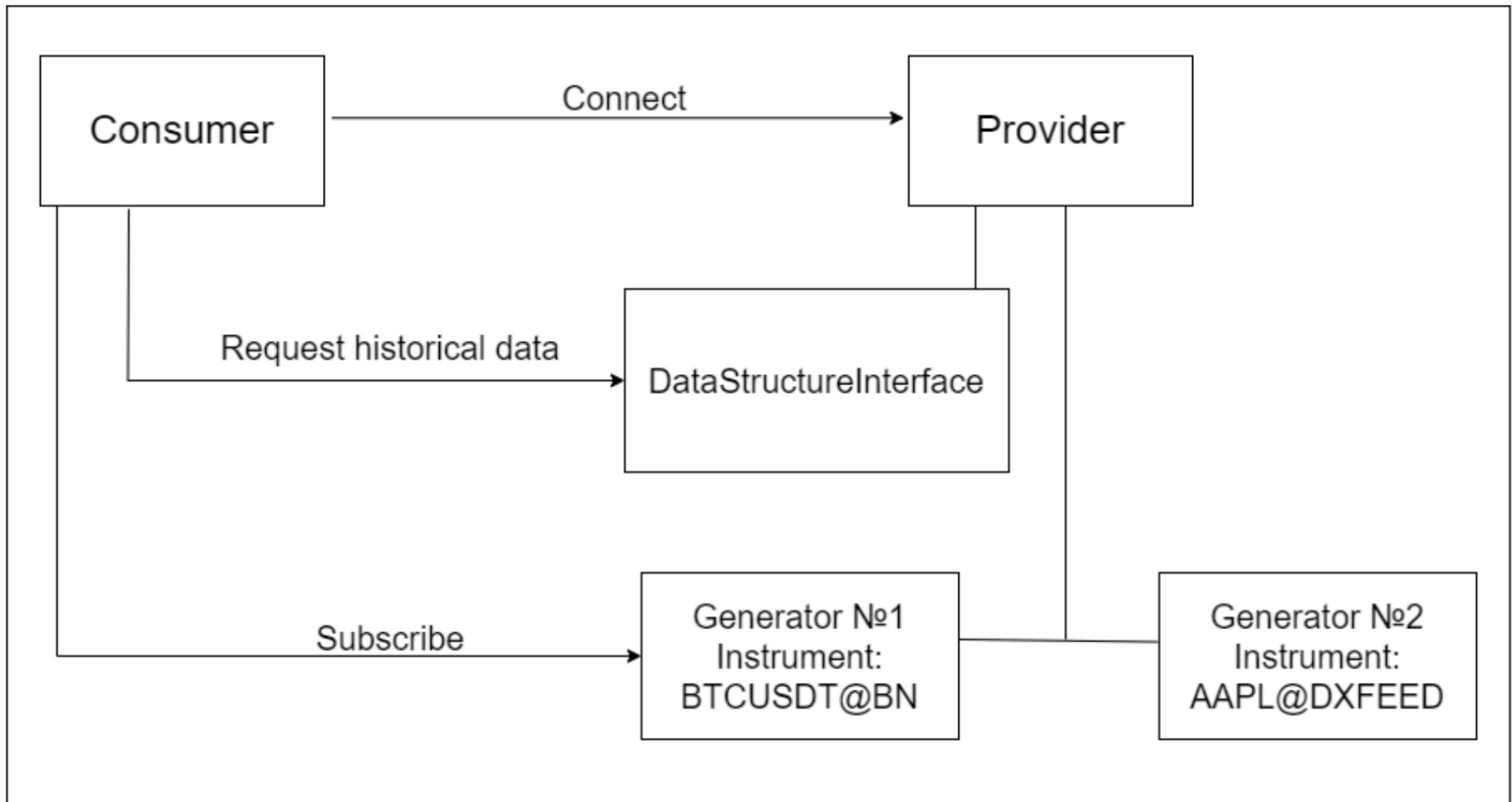


BrAPI Overview

After the consumer add-on and the provider add-on have started, the consumer should connect to the provider. If the connection is successful, the consumer can subscribe to live events of one of the provider's generators and receive all of its events live, or request historical data for a certain period(Java API only).

Generators

One provider can have many generators. Quite often they work as one generator per alias, but not always, and it can be that one alias can have more than one generator, and a generator can work on more than one alias. In the example below, the provider has two generators for two different instruments. The consumer connects to the provider and subscribes to one of the generators. Once subscribed, the consumer will receive all live events from that generator. Additionally, the consumer can request historical data for a specific period using the provider's `DataSetInterface`.



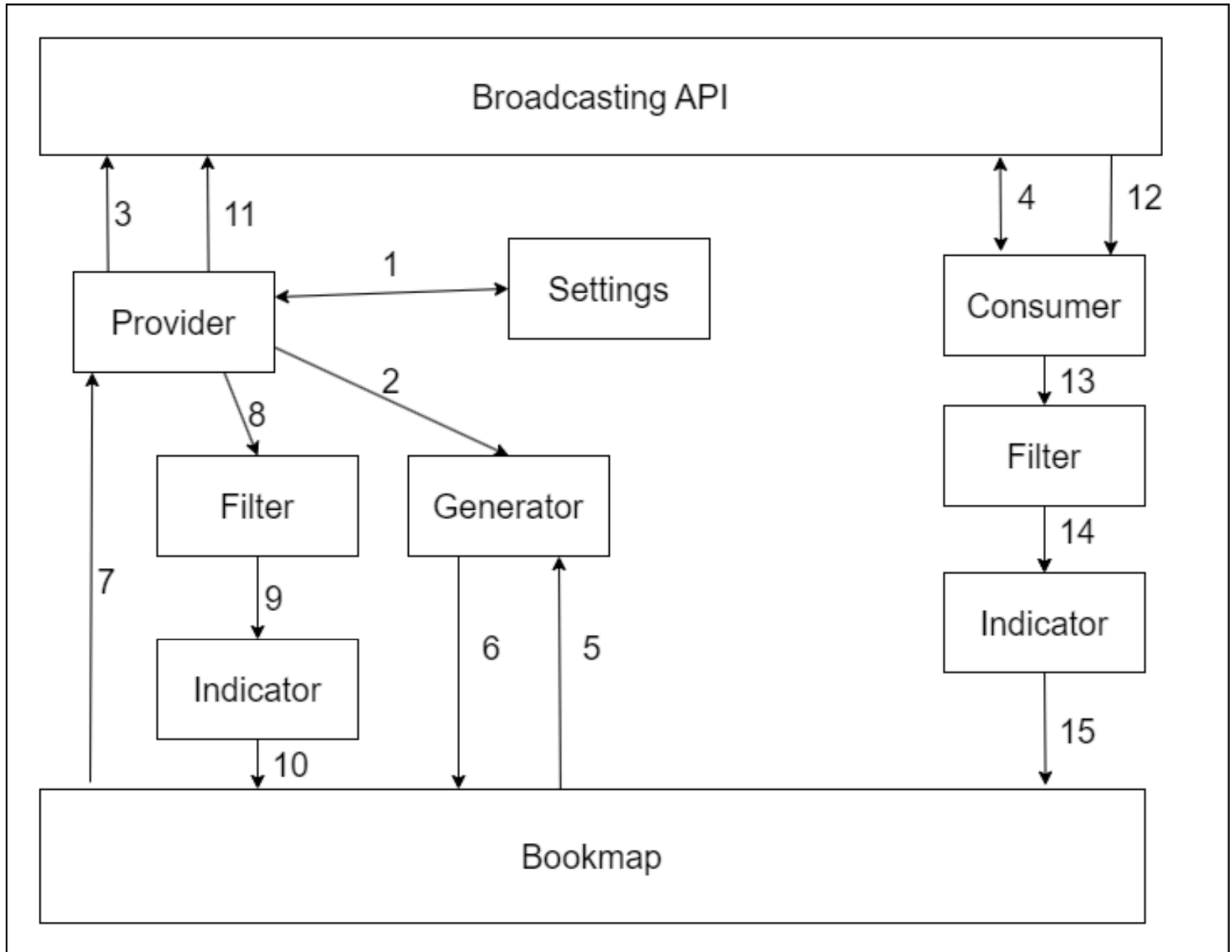
Filter and Settings

The generator may have a filter and settings. That is, if you are a provider, the **broadcast** settings and filter are optional. The filter is used to filter out unimportant events and keep only important events. Usually the generator **broadcasts** all events (raw events), but for example it only visualises or sends an alert to events that crossed its threshold (filtered events). So, if you want to receive only important data, a filter will help you. By passing raw live events through the filter you will get data that the generator thinks is important.

The settings is a provider class with all generator or add-on parameters. It can contain both parameters, on the basis of which the generator calculates data, and parameters not related to the generator, for example, the colour for the indicator. This object is passed only so that the consumer knows what parameters the provider is working with.

Java BrAPI Overview

Here's a step-by-step description of how the generator works and the role of the filter and settings.



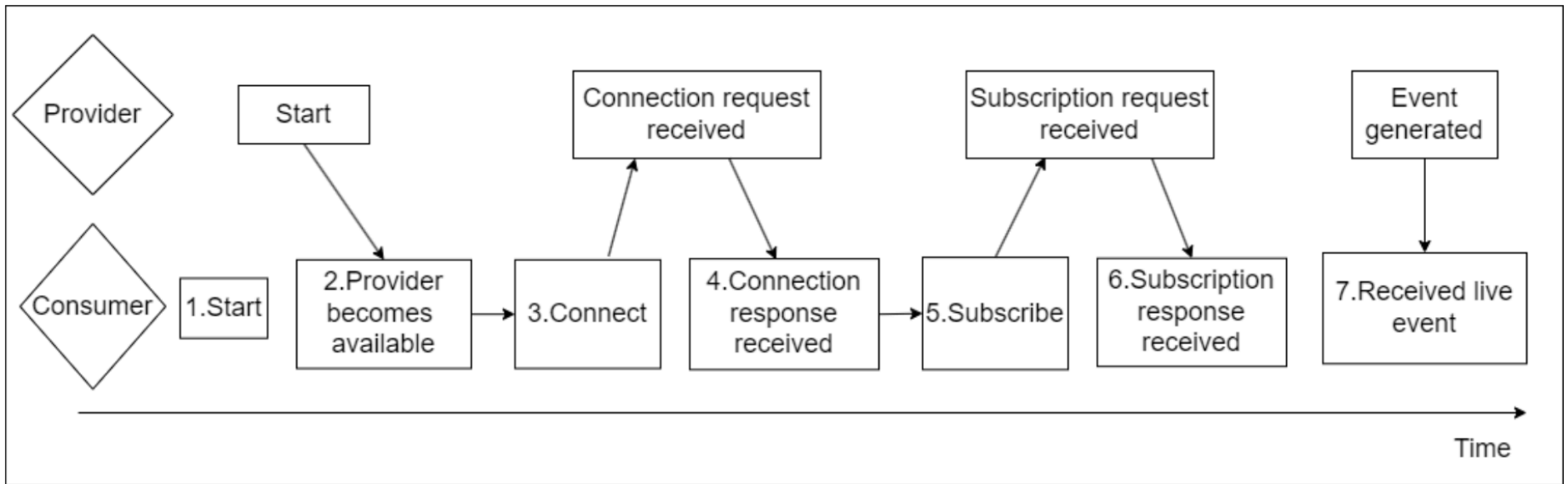
1. Settings. A class with all the parameters of a generator or add-on.
2. Generator creation. Create a StrategyUpdateGenerator implementation and register it in the BM using Layer1ApiUserMessageAddStrategyUpdateGenerator.
3. The provider registers the generator, settings and filter in BrAPI.
4. The consumer subscribes to the generator, and they can now get the filter and settings from GeneratorInfo.
5. The generator receives data from Bookmap, for example trade or depth events, from which it generates its events.
6. The generator sends raw generated events to Bookmap.
7. Bookmap sends raw events to the add-on provider.
8. The provider sends raw events to the filter. To get filtered events, for example, that crossed a threshold, the add-on will draw on important events or react to them with a sound notification.
9. The provider sends filtered events to Indicator.
10. The indicator visualises the filtered events.
11. The provider sends raw events to BrAPI.
12. The consumer receives raw events from BrAPI.
13. The consumer sends raw events to filter.
14. The consumer sends filtered events to Indicator.
15. The indicator visualises the filtered events.

How to use it as a consumer.

We have two demo add-ons that are designed to show you how to work with BrAPI as a consumer: Simple Demo Consumer and Demo Consumer. You should start with the Simple Demo Consumer to understand the basics more easily.

Subscription to live events

Here's a step-by-step breakdown of what you should do to get live events from a provider.



1. **Start:** In the first step, we get an instance of **BroadcasterConsumer** from **BroadcastFactory.getBroadcasterConsumer()**. We will interact with BrAPI through this interface. Then we start BrAPI with the **broadcaster.start()** method. It is best to do it after you receive **UserMessageLayersChainCreatedTargeted** in **onUserMessage()**.
2. **Provider becomes available:** The provider must be available for you to connect to it. For example, a user will load a provider add-on in Bookmap later than your add-on, then you can find out when the provider becomes available using **ProviderStatusListener**. You can register **ProviderStatusListener** in **BroadcasterConsumer**, when any provider becomes available the **providerBecameAvailable** method will be called.
3. **Connect:** Use the **connectToProvider** method from **BroadcasterConsumer** to connect to the provider. Use the full name of the provider, for example **"velox.indicators.absorption.AbsorptionIndicator"**, you can get the full provider name in the **getAvailableProviders()** method which will return a list of all available providers.
4. **Connection response received:** You will need to set the **ConnectionStatusListener** in the **connectToProvider** method. This listener will receive the result of connection. If we receive **true** in this listener we will know that we are connected and can work further.
5. **Subscribe:** We can get a list of all available generators of the provider from the **getGeneratorsInfo** method. Then take the name of the generator we need and subscribe to it using the **subscribeToLiveData** method. In this method we will need to set the name of

the provider, the name of the generator, LiveEventListener to receive live events and LiveConnectionStatusListener(optional, can be set null) to receive the status of our subscription.

6. Subscription response received: If we set the LiveConnectionStatusListener when we subscribed, then getting true in this listener will let us know that we have successfully subscribed.
7. Received live event: After we have successfully subscribed, we will receive live events in LiveEventListener. We will receive an Object that needs to be cast to the correct event provider class using CastUtilities.

Request for historical events.

After you have successfully connected to a provider you can request the history for some period for a specific generator.

You need to take into careful attention that BrDataStructureInterface has 4 methods, 2 of which return aggregated data and 2 non-aggregated data. Some providers return only aggregated data, others only non-aggregated data. You can find out which data your provider works with in the description of providers.

To do this, get BrDataStructureInterface from the getDataStructureInterface(String providerName) method. Then use one of its methods:

1. List<BrDataStructureInterface.TreeResponseInterval> get(Class<?> strategyClass, String generatorName, long t0, long intervalWidth, int intervalNumber, String alias, Class<?>[] customEvents). Aggregated data. Returns a list with aggregated events, where number of events = intervalWidth, time of first event = t0, how much time one aggregated event covers = intervalWidth.
2. BrDataStructureInterface.TreeResponseInterval get(Class<?> strategyClass, String generatorName, long t1, String alias, Class<?>[] customEvents). Aggregated data. Returns a single aggregated event from the first event to t1.
3. List<Object> get(Class<?> strategyClass, String generatorName, long t0, long t1, String alias). Non-aggregated data. Returns all events from t0 to t1.
4. List<? extends Event> get(Class<?> strategyClass, String generatorName, long t1, String alias). Non-aggregated data. Returns all events from the first event to t1.

The result of the historical events query should also be cast using `CastUtilities`.

Listeners

You can use these listener interfaces to create an implementation of them and install them in BrAPI. Then when a certain event happens, the listener method will be called.

1. `ProviderStatusListener`, can be set in `BroadcasterConsumer`. This listener has such methods:
 - `providerBecameAvailable(String providerName, String providerId)`, will be called when the provider becomes available.
 - `providerBecameUnavailable(String providerName, String providerId)`, will be called when the provider becomes unavailable.
 - `providerUpdateGenerator(String providerName, String providerId, GeneratorInfo generator, boolean isOnline)`, will be called when a generator of the provider becomes available or unavailable.
2. `ConnectionStatusListener`, can be set in the `connectToProvider()` method in `BroadcasterConsumer`. The listener will be called when the connection is established or interrupted.
3. `LiveConnectionStatusListener`, can be set in the `subscribeToLiveData()` method in `BroadcasterConsumer`. The listener will be called when the subscription is established or interrupted.
4. `LiveEventListener`, can be set in the `subscribeToLiveData()` method in `BroadcasterConsumer`. The listener will receive each live event generated by the provider's generator.
5. `UpdateFilterListener`, can be set in the `setFilterListener()` method in `GeneratorInfo`. The listener will receive the filter after the provider's generator updates it.
6. `UpdateSettingsListener`, can be set in the `setSettingsListener()` method in `GeneratorInfo`. The listener will receive the settings after the provider's generator updates it.

CastUtilities

We will get Object in listeners such as `LiveEventListener`, `UpdateFilterListener` and `UpdateSettingsListener`. You have two options for what to do with such objects:

- Add the provider module to your dependencies. These modules contain all classes that the provider **broadcasts**, and you can find the provider module in the providers section. Now you have a class and you can use CastUtilities. An example where we cast Object to Adsorption Indicator event:

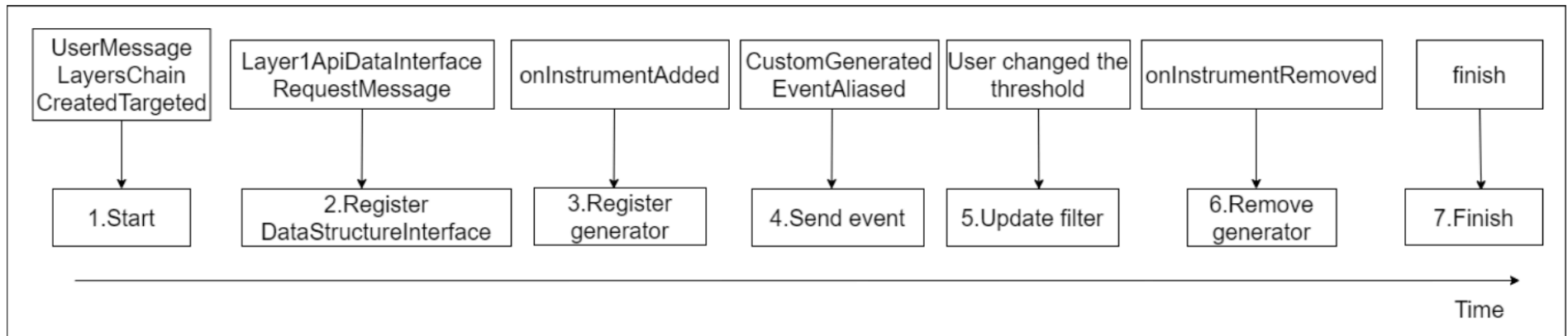
```
@Override
public void giveEvent(Object o) {
    try {
        EventInterface eventInterface = CastUtilities.castObject(o, TradeEvent.class);
        events.addLast(eventInterface.toString());
    } catch (FailedToCastObject e) {
        throw new RuntimeException(e);
    }
}
```

- You don't have to add a module to your dependencies; know what fields the class has and get them using the Reflection API.

How to use it as a provider.

We have a Demo Simple Provider add-on that implements BrAPI as a provider. Demo Simple Provider creates a generator on each alias, calculates moving average based on trades and **broadcasts** these events live.

Here's a step-by-step description of how a provider can **broadcast** events.



1. Start: In the first step, we get an instance of **BroadcasterProvider** from **BroadcastFactory.getBroadcasterProvider(layer1ApiProvider, providerName, providerMainClass)**. We will interact with BrAPI through this interface. Then we start BrAPI with the **broadcaster.start()** method. It is best to do it after you receive **UserMessageLayersChainCreatedTargeted** in **onUserMessage**.
2. Register **DataStructureInterface**: It is optional if you want to **broadcast** historical data. By sending a **Layer1ApiDataInterfaceRequestMessage** to the BM you will get your **DataStructureInterface**. Create your **BrDataStructureInterface** implementation in which you will request data from your **DataStructureInterface**. Register your implementation to the **BrDataStructureInterface** using the **broadcaster.registerDataStructureInterface()** method.
3. Register generator: You can have anything in the generator role. However, we strongly recommend using a BM generator (your **StrategyUpdateGenerator** implementation that you register in BM using **Layer1ApiUserMessageAddStrategyUpdateGenerator**). Register the generator in BrAPI using the **broadcaster.registerGenerator(GeneratorName, eventClass, Filter(can be null), Settings(can be null))** method.
4. Send live event: You can get live events generated by your generator in **onUserMessage(Object data)** by checking if data is a **CustomGeneratedEventAliased** instance. Then send this event to BrAPI using **sendLiveEvent(GeneratorName, event)** method.
5. Update filter: It is optional if you want to **broadcast** some kind of filter to consumers. If you want to **broadcast** a filter or settings, after their parameters have changed you can send the updated filter and settings using the **updateFilter(GeneratorName, filter)** and **updateSettings(GeneratorName, settings)** methods.
6. Remove generator: Before you remove your generator from the BM by sending **Layer1ApiUserMessageAddStrategyUpdateGenerator** with **isAdd = false**. Remove your generator from the BrAPI using the

broadcaster.registerGenerator(GeneratorName) method.

7. Finish: In the finish() method of your add-on, terminate the BrAPI using the **broadcaster.finish()** method.

Listeners

You can use these listener interfaces to create an implementation of them and install them in BrAPI. Then when a certain event happens, the listener method will be called.

- **ConnectionToConsumersStatusListener**, can be set in **BroadcasterProvider**. This will be called when some consumer connects or disconnects from your add-on.
- **SubscriptionToConsumersStatusListener**, can be set in **BroadcasterProvider**. This will be called when some consumer subscribes or unsubscribes from your add-on generator.

Current BrAPI providers.

This section describes all BM add-ons that **broadcast** data via BrAPI.

Absorption Indicator and Sweeps Indicator

The Absorption and Sweeps indicators **broadcast** trades which contain all necessary information to mark them as absorptions or sweeps accordingly.

Each instrument has a separate generator, and the generator name is equal to the instrument alias. The generator is recalculated if the **timeLimitMillis** or **minLevels** parameter is changed, or if historical data for the instrument was loaded.

Events are **broadcasted** and put into the generator with delay, and the size of the delay is not less than the **timeLimitMillis** parameter in settings.

Events:

- Trade - trade event that contains time, price, size and isBid parameters. The main parameter to look at is maxChainSize, which determines the total size of the biggest by volume chain of trades which includes this trade with the current settings.

Settings: ProviderSettingsProxy - object with all add-on settings. The most important parameters are:

- sizeLimit - parameter used for trades filtering, see Filter description
- timeLimitMillis - maximum time between trades in a trades chain used for calculations, in milliseconds
- minLevels - minimum number of price levels on which trades of the same chain should happen to be considered a sweep. Has no meaning for absorptions.

Other parameters:

- isEnabled - shows if the add-on is enabled for the specified instrument
- alias - name of the instrument
- alertEnabled - shows if the sound alert is enabled for the instrument
- alertTemplate - template of the sound alert
- priceMatters - legacy parameter, always set to true for Absorption and to false for Sweep indicator
- autoMode - shows that the sizeLimit is calculated automatically when true
- sdIntervalMinutes - interval of reading backfill data for calculating sizeLimit in auto mode
- sdMultiplier - multiplier used in the formula for sizeLimit calculation in auto mode

Filter:

- Filter object contains single field sizeLimit. If trade.maxChainSize $\geq$ sizeLimit we consider the trade as a part of absorption/sweeps chain of trades.

Stops&Icebergs On-Chart

SI On-Chart **broadcasts** events related to detected stops and icebergs, and each instrument has a separate generator. The generator name is equal to instrument alias.

Events:

- StopEvent - indicates a stop trade including orderID of a stop order, its price, size and time. isBid parameter indicates if it is buy or sell, totalSize indicates total size of all the trades in the stop trades chain.
- IcebergEvent - the same for icebergs, but events could be of a different type (detection, trade, movement, execution, cancellation). Size could be non-zero only for trades and detections. For detection event size includes sizes of all trades the iceberg had before being detected. Execution event indicates that the iceberg was fully executed, cancellation means that the iceberg was cancelled after partial execution. Both these events have a totalSize parameter which shows total volume of trades for this iceberg at the moment the event happened. Movement event indicates that the iceberg was moved to another price level.

Settings:

- SitSettings - object with all add-on settings

Filter:

- Threshold filter - contains values of size thresholds for displaying icebergs and stops on the chart at the moment.

Market Pulse

Market Pulse **broadcasts** the value of widgets. In Market Pulse, each running widget is a separate generator. If it is a widget with two bars, the generator is **broadcasted** - MPDoubleBarEvent, if it is a widget with one bar, the generator is **broadcasted** - MPSingleBarEvent. The generator name consists of the algorithm name and a unique generator identifier, for example absorptionPressure;1685189706.

Events:

- **MPSingleBarEvent**: contains value - absolute value and estimate - with relative value to the maximum value for the training period.
- **MPDoubleBarEvent**: contains absolute values of buyers and sellers and estimated values of buyers and sellers - with relative value to maximum, MaxValue - maximum value for the training period.

Settings:

- **MPWidgetSettings**: an object with all widget settings, including: id - unique widget id, name - widget name, modelId - algorithm name, params - widget parameters (such as half-life period, training period, etc.), aliases, event class (MPSingleBarEvent or MPDoubleBarEvent).

Filter:

- **MPFilter**: filters the value based on the threshold specified in the widget. If the event value is greater than the threshold, this event is returned, if it is less, null is returned.

Historical data: aggregated data.

Breakeven Point

Breakeven Point **broadcasts** the breakeven price based on open orders.

Events:

- **BreakevenEvent**: contains the breakeven price. Event sends when a consumer subscribes and when the breakeven price changes.
- **PaidCommissionEvent**: contains the type of commission and how much was paid. The event is sent after each order is executed.

Settings:

- **CommissionsSettings**: this settings class describes how many commissions are currently set and of what type.

Filter:

- No filter required.

Historical data: aggregated data and non-aggregated data.

Strength level add-on

This uses one generator for each instrument and **broadcasts** BrIcebergEvent events. The name of the generator consists of the name of the add-on and the instrument (for example, 'Strength level addon::BTCUSDT@BN').

Event:

- BrIcebergEvent: contains the following parameters: level - the price of the event, size - the size of the event and, accordingly, isBid - indicates the side of the event, mode - we detect events in two modes at once, so this field is needed to understand for which mode the event was found.

Settings:

- ProviderSettingsProxy: maxPostTradeIncreaseDelay and minPostTradeIncreaseSize - we specify these values in the add-on settings (they are used for filtration).

Filter:

- Filters events based on mode, delay, and size. If the event's mode matches the one selected in the settings, the delay is less than the one specified in the settings, and the size is greater than the one specified in the settings, this event will be returned, otherwise null.

Anchored VWAP

AVWAP broadcasts the VWAP and band (standard deviation) values of each line. A new generator is registered for each new VWAP line. The name of the generator consists of a sequence of 30 random letters, so to understand whose generator it is, see the settings (ProviderSettingsProxy).

Event:

- BrVwapEvent: contains the VWAP value, standard deviation, and time. Settings:
- ProviderSettingsProxy: username - line name, alias, anchor - start time of this line, brResetPointInfoList - a list with brief information about resets, brBandInfoList - contains the coefficients of active bands.

Filter:

- No filters.

FAQ

1. How can I create an add-on or adapter in languages other than Java?

- Using the [Python API](#), you can develop L1 add-ons in Python. Other OS interprocess communication methods, including web sockets and JNI, are also supported. However, you can load a minimal Java module as a proxy agent.

2. How does Bookmap handle order routing and market data?

- Bookmap doesn't route orders or real-time market data through its servers*, it acts as a front-end connecting directly to the destination using user-configured credentials.
- *The exception is Bookmap Data connections, our custom solution.

3. What trading strategies can be designed using the Bookmap API?

- There are no restrictions. Implement any business logic or complex orders management. However, always be aware of risks, especially those associated with latency. Traders must always consider the worst case scenarios in regards to their orders.

4. What are semi-automatic trading strategies in Bookmap?

- Bookmap API allows trading strategies to “intercept” and manage / modify orders, sent by the user from the chart. This mode may also appear as “inject” mode. It can be useful, for instance, for randomizing the take-profit & stop-loss distances. Another example is a risk module, Note, however, that Bookmap expects valid feedback about the status of all orders sent from the chart. It’s the responsibility of such a strategy to provide this feedback.

5. Can I use real-time data for API development?

- For Crypto, real-time data is available. However, for stock and future API developments you should use delayed data due to vendor and exchange limitations.

6. Can I use the developed products for non-display purposes?

- Stock and future developments are solely for display. For non-display use of stock or future data, acquire the necessary license from the relevant data vendor.

For additional queries, please email support@bookmap.com.

*Last updated on **Jun 25, 2025***